

Algorithms – Data Structures

Exercise 1: Inventory Management System

Understanding the Problem and Data Structures and Algorithms Importance

In a warehouse inventory system, there may be thousands or millions of products.

Data Structures help:

- Store products efficiently.
- Retrieve product details quickly.
- Update stock information easily.
- Delete discontinued products efficiently.

Algorithms help:

- Search products quickly.
- Sort products based on price, quantity, etc.
- Manage inventory operations efficiently.

Without proper data structures and algorithms, operations become slow as the inventory grows.

Suitable Data Structures

Data Structure	Advantages	Disadvantages
ArrayList	Easy to use, dynamic size	Searching is $O(n)$
LinkedList	Fast insertion/deletion	Slow searching
HashMap	Fast add, search, update, delete	More memory usage

Best Choice: HashMap

Setup

Product.java

```
class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
}
```

```

    public String toString() {
        return "ID: " + productId +
            ", Name: " + productName +
            ", Quantity: " + quantity +
            ", Price: ₹" + price;
    }
}

```

InventoryManagement.java

```

import java.util.HashMap;

class InventoryManagement {

    HashMap<Integer, Product> inventory = new HashMap<>();
    void addProduct(Product p) {
        inventory.put(p.productId, p);
        System.out.println("Product Added.");
    }
    void updateProduct(int id, int quantity, double price) {
        Product p = inventory.get(id);

        if (p != null) {
            p.quantity = quantity;
            p.price = price;
            System.out.println("Product Updated.");
        } else {
            System.out.println("Product Not Found.");
        }
    }
    void deleteProduct(int id) {
        if (inventory.remove(id) != null) {
            System.out.println("Product Deleted.");
        } else {
            System.out.println("Product Not Found.");
        }
    }
    void displayProducts() {
        for (Product p : inventory.values()) {
            System.out.println(p);
        }
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {

        InventoryManagement ims = new InventoryManagement();

        Product p1 = new Product(101, "Laptop", 20, 50000);
        Product p2 = new Product(102, "Mouse", 100, 500);
        Product p3 = new Product(103, "Keyboard", 50, 1200);
    }
}

```

```

ims.addProduct(p1);
ims.addProduct(p2);
ims.addProduct(p3);

System.out.println("\nInventory:");
ims.displayProducts();

System.out.println("\nUpdating Product 102...");
ims.updateProduct(102, 120, 550);

System.out.println("\nDeleting Product 103...");
ims.deleteProduct(103);

System.out.println("\nFinal Inventory:");
ims.displayProducts();
}
}

```

Output:

```

PS C:\Users\Madhu\Desktop\cts java\dsa\InventoryManagementSystem-1> javac *.java
PS C:\Users\Madhu\Desktop\cts java\dsa\InventoryManagementSystem-1> java Main
Product Added.
Product Added.
Product Added.

Inventory:
ID: 101, Name: Laptop, Quantity: 20, Price: ₹50000.0
ID: 102, Name: Mouse, Quantity: 100, Price: ₹500.0
ID: 103, Name: Keyboard, Quantity: 50, Price: ₹1200.0

Updating Product 102...
Product Updated.

Deleting Product 103...
Product Deleted.

Final Inventory:
ID: 101, Name: Laptop, Quantity: 20, Price: ₹50000.0
ID: 102, Name: Mouse, Quantity: 120, Price: ₹550.0
❖ PS C:\Users\Madhu\Desktop\cts java\dsa\InventoryManagementSystem-1>

```

4. Analysis: Time Complexity using HashMap

Operation	Method Used	Time Complexity
Add Product	put()	O(1)
Search Product	get()	O(1)
Update Product	get() + update	O(1)
Delete Product	remove()	O(1)
Display Products	Traversal	O(n)

Where **n** = number of products.

Optimization

1. Use HashMap.
2. Unique Product Id
3. Store Frequently Accessed Data
4. Use Database for Very Large Inventories

Exercise 2: E-commerce Platform Search Function

Understanding Asymptotic Notation

Big O Notation:

Big O Notation is used to measure the efficiency of an algorithm as the input size increases.

It describes how the running time grows with the number of elements (**n**).

Examples

Complexity	Meaning
$O(1)$	Constant Time
$O(\log n)$	Logarithmic Time
$O(n)$	Linear Time
$O(n^2)$	Quadratic Time

Why is Big O Important?

- Helps compare algorithms.
- Predicts performance for large datasets.
- Helps choose the most efficient solution.

For an e-commerce platform with thousands of products, choosing an efficient search algorithm is very important.

Search Operation Cases

Best Case

Element found at the first position.

Product found immediately.

Time Complexity:

$O(1)$

Average Case

Element found somewhere in the middle.

Time Complexity:

$O(n)$

for Linear Search.

$O(\log n)$

for Binary Search.

Worst Case

Element is:

- At the last position
- Or not present

Time Complexity:

Linear Search $\rightarrow O(n)$

Binary Search $\rightarrow O(\log n)$

Setup:

Product.java

```
class Product {  
  
    int productId;  
    String productName;  
    String category;  
  
    Product(int productId, String productName, String category) {  
        this.productId = productId;  
        this.productName = productName;  
        this.category = category;  
    }  
  
    public String toString() {  
        return productId + " " + productName + " " + category;  
    }  
}
```

Implementation

SearchDemo.java

```
public class SearchDemo {  
  
    // Linear Search  
    static Product linearSearch(Product[] products, int targetId) {  
  
        for (Product p : products) {  
            if (p.productId == targetId) {  
                return p;  
            }  
        }  
    }  
}
```

```

    }
}

return null;
}

// Binary Search
static Product binarySearch(Product[] products, int targetId) {

    int left = 0;
    int right = products.length - 1;

    while (left <= right) {

        int mid = (left + right) / 2;

        if (products[mid].productId == targetId)
            return products[mid];

        else if (products[mid].productId < targetId)
            left = mid + 1;

        else
            right = mid - 1;
    }

    return null;
}

public static void main(String[] args) {

    Product[] products = {
        new Product(101, "Laptop", "Electronics"),
        new Product(102, "Mouse", "Electronics"),
        new Product(103, "Keyboard", "Electronics"),
        new Product(104, "Headphones", "Electronics"),
        new Product(105, "Monitor", "Electronics")
    };

    int searchId = 104;

    Product result1 = linearSearch(products, searchId);

    if (result1 != null)
        System.out.println("Linear Search Found: " + result1);
    else
        System.out.println("Product Not Found");

    Product result2 = binarySearch(products, searchId);

    if (result2 != null)
        System.out.println("Binary Search Found: " + result2);
    else
        System.out.println("Product Not Found");
}

```

}

Output:

Working of Linear Search

Suppose we search:

104

Array:

101 102 103 104 105

Checks:

101 -No
102 - No
103 - No
104 -Yes

Elements checked = 4

Working of Binary Search

Array must be sorted.

101 102 103 104 105

Search:

104

Steps:

Mid = 103
104 > 103

Move Right

104 found

Only 2 comparisons needed.

4. Analysis

Time Complexity Comparison

Operation	Linear Search	Binary Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$

Example with 1,000,000 Products

Linear Search

May check:

1,000,000 products

Complexity:

$O(n)$

Binary Search

Checks approximately:

$\log_2(1,000,000)$
 ≈ 20 comparisons

Complexity:

$O(\log n)$

Much faster.

Which Algorithm is Better?

Linear Search

Advantages:

- Simple implementation.

- Works on unsorted data.

Disadvantages:

- Slow for large datasets.

Binary Search

Advantages:

- Very fast.
- Suitable for large product catalogs.

Disadvantages:

- Requires sorted data.

For an **E-commerce Platform**, **Binary Search** is the better choice .

Exercise 3: Sorting Customer Orders

1. Understanding Sorting Algorithms

Sorting means arranging data in a particular order (ascending or descending).

In an e-commerce platform, sorting orders by totalPrice helps identify high-value orders quickly.

Bubble Sort

Working

- Compare adjacent elements.
- Swap if they are in the wrong order.
- Repeat until the array is sorted.

Example:

5000 3000 7000

5000 > 3000 → Swap

3000 5000 7000

Time Complexity

Case	Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

Insertion Sort

Working

- Assume first element is sorted.
- Insert each new element into its correct position.

Example:

5 3 4

5

3 5

3 4 5

Time Complexity

Case Complexity

Best $O(n)$

Average $O(n^2)$

Worst $O(n^2)$

Quick Sort

Working

- Select a pivot element.
- Place smaller elements on the left.
- Place larger elements on the right.
- Recursively sort both parts.

Example:

50 30 70 20 60

Pivot = 50

20 30 | 50 | 60 70

Time Complexity

Case Complexity

Best $O(n \log n)$

Average $O(n \log n)$

Worst $O(n^2)$

Merge Sort

Working

- Divide array into halves.
- Sort each half.
- Merge them.

Example:

8 3 6 2

8 3 | 6 2

3 8 | 2 6

2 3 6 8

Time Complexity

Case Complexity

Best $O(n \log n)$

Average $O(n \log n)$

Worst $O(n \log n)$

Setup

Order.java

```
class Order {  
  
    int orderId;  
    String customerName;  
    double totalPrice;  
  
    Order(int orderId, String customerName, double totalPrice) {  
        this.orderId = orderId;  
        this.customerName = customerName;  
        this.totalPrice = totalPrice;  
    }  
  
    public String toString() {  
        return "Order ID: " + orderId +  
            ", Customer: " + customerName +  
            ", Total Price: ₹" + totalPrice;  
    }  
}
```

Implementation

SortingOrders.java

```
public class SortingOrders {  
  
    // Bubble Sort  
    static void bubbleSort(Order[] orders) {  
  
        int n = orders.length;  
  
        for (int i = 0; i < n - 1; i++) {  
  
            for (int j = 0; j < n - i - 1; j++) {  
  
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {  
  
                    Order temp = orders[j];  
                    orders[j] = orders[j + 1];  
                    orders[j + 1] = temp;  
                }  
            }  
        }  
    }  
}
```

```
    }  
  }  
}
```

// Quick Sort

```
static void quickSort(Order[] orders, int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(orders, low, high);
```

```
        quickSort(orders, low, pi - 1);
```

```
        quickSort(orders, pi + 1, high);
```

```
    }
```

```
}
```

```
static int partition(Order[] orders, int low, int high) {
```

```
    double pivot = orders[high].totalPrice;
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (orders[j].totalPrice < pivot) {
```

```
            i++;
```

```
            Order temp = orders[i];
```

```
            orders[i] = orders[j];
```

```
            orders[j] = temp;
```

```
        }
```

```
    }
```

```
    Order temp = orders[i + 1];
```

```
    orders[i + 1] = orders[high];
```

```
    orders[high] = temp;
```

```
    return i + 1;
```

```
}
```

```
static void display(Order[] orders) {
```

```
    for (Order o : orders) {
```

```
        System.out.println(o);
```

```
    }
```

```
}
```

```
public static void main(String[] args) {
```

```
    Order[] orders1 = {
```

```
        new Order(101, "Rahul", 5000),
```

```
        new Order(102, "Priya", 2000),
```

```
        new Order(103, "Arun", 8000),
```

```
        new Order(104, "Meena", 3000)
```

```
    };
```

```

System.out.println("Before Bubble Sort:");
display(orders1);

bubbleSort(orders1);

System.out.println("\nAfter Bubble Sort:");
display(orders1);

Order[] orders2 = {
    new Order(101, "Rahul", 5000),
    new Order(102, "Priya", 2000),
    new Order(103, "Arun", 8000),
    new Order(104, "Meena", 3000)
};

quickSort(orders2, 0, orders2.length - 1);

System.out.println("\nAfter Quick Sort:");
display(orders2);
}
}

```

Output:

```

• PS C:\Users\Madhu\Desktop\cts java\dsa\Sorting Customer Orders-3> javac *.java
• PS C:\Users\Madhu\Desktop\cts java\dsa\Sorting Customer Orders-3> java SortingOrders
Before Bubble Sort:
Order ID: 101, Customer: Rahul, Total Price: ₹5000.0
Order ID: 102, Customer: Priya, Total Price: ₹2000.0
Order ID: 103, Customer: Arun, Total Price: ₹8000.0
Order ID: 104, Customer: Meena, Total Price: ₹3000.0

After Bubble Sort:
Order ID: 102, Customer: Priya, Total Price: ₹2000.0
Order ID: 104, Customer: Meena, Total Price: ₹3000.0
Order ID: 101, Customer: Rahul, Total Price: ₹5000.0
Order ID: 103, Customer: Arun, Total Price: ₹8000.0

After Quick Sort:
Order ID: 102, Customer: Priya, Total Price: ₹2000.0
Order ID: 104, Customer: Meena, Total Price: ₹3000.0
Order ID: 101, Customer: Rahul, Total Price: ₹5000.0
Order ID: 103, Customer: Arun, Total Price: ₹8000.0
❖ PS C:\Users\Madhu\Desktop\cts java\dsa\Sorting Customer Orders-3>

```

Analysis

Time Complexity Comparison

Case	Bubble Sort	Quick Sort
------	-------------	------------

Case	Bubble Sort	Quick Sort
Best Case	$O(n)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n^2)$

Why Quick Sort is Preferred?

Bubble Sort

Too many comparisons
Slow for large datasets
Not suitable for e-commerce platforms

Quick Sort

Fast average performance
 $O(n \log n)$ complexity
Handles large order data efficiently
Widely used in real-world applications

Preferred Algorithm

✓Quick Sort

Exercise 4: Employee Management System

Understanding Array Representation

Arrays are Represented As

An array stores elements in **contiguous (continuous) memory locations**.

Example:

Index: 0 1 2 3
Array: Emp1 Emp2 Emp3 Emp4

Memory Representation:

1000 1004 1008 1012
Emp1 Emp2 Emp3 Emp4

Each element occupies consecutive memory locations.

Advantages of Arrays

1. Fast Access
2. Easy Traversal
3. Less Memory Overhead

2. Setup

Employee.java

```
class Employee {  
  
    int employeeId;  
    String name;  
    String position;  
    double salary;  
  
    Employee(int employeeId, String name,  
             String position, double salary) {  
  
        this.employeeId = employeeId;  
        this.name = name;  
        this.position = position;  
        this.salary = salary;  
    }  
  
    public String toString() {  
        return "ID: " + employeeId +  
            ", Name: " + name +  
            ", Position: " + position +
```

```
        ", Salary: ₹" + salary;
    }
}
```

3. Implementation

EmployeeManagement.java

```
class EmployeeManagement {

    Employee[] employees;
    int count;

    EmployeeManagement(int size) {
        employees = new Employee[size];
        count = 0;
    }

    // Add Employee
    void addEmployee(Employee e) {

        if (count < employees.length) {
            employees[count] = e;
            count++;
            System.out.println("Employee Added");
        } else {
            System.out.println("Array Full");
        }
    }

    // Search Employee
    Employee searchEmployee(int id) {

        for (int i = 0; i < count; i++) {

            if (employees[i].employeeId == id) {
                return employees[i];
            }
        }

        return null;
    }

    // Traverse Employees
    void traverseEmployees() {

        for (int i = 0; i < count; i++) {
            System.out.println(employees[i]);
        }
    }

    // Delete Employee
    void deleteEmployee(int id) {

        int index = -1;
```

```

    for (int i = 0; i < count; i++) {

        if (employees[i].employeeId == id) {
            index = i;
            break;
        }
    }

    if (index == -1) {
        System.out.println("Employee Not Found");
        return;
    }

    for (int i = index; i < count - 1; i++) {
        employees[i] = employees[i + 1];
    }

    employees[count - 1] = null;
    count--;

    System.out.println("Employee Deleted");
}
}

```

Main.java

```

public class Main {

    public static void main(String[] args) {

        EmployeeManagement em =
            new EmployeeManagement(10);

        em.addEmployee(
            new Employee(101,
                "Rahul",
                "Manager",
                60000));

        em.addEmployee(
            new Employee(102,
                "Priya",
                "Developer",
                50000));

        em.addEmployee(
            new Employee(103,
                "Arun",
                "Tester",
                40000));

        System.out.println("\nEmployee Records:");

        em.traverseEmployees();

        System.out.println("\nSearching Employee 102:");
    }
}

```

```

Employee e =
    em.searchEmployee(102);

if (e != null)
    System.out.println(e);
else
    System.out.println("Not Found");

System.out.println("\nDeleting Employee 102");

em.deleteEmployee(102);

System.out.println("\nUpdated Records:");

em.traverseEmployees();
}
}

```

Output:

```

PS C:\Users\Madhu\Desktop\cts java\dsa\Employee Management System-4> java Main
Employee Added
Employee Added
Employee Added

Employee Records:
ID: 101, Name: Rahul, Position: Manager, Salary: ₹60000.0
ID: 102, Name: Priya, Position: Developer, Salary: ₹50000.0
ID: 103, Name: Arun, Position: Tester, Salary: ₹40000.0

Searching Employee 102:
ID: 102, Name: Priya, Position: Developer, Salary: ₹50000.0

Deleting Employee 102
Employee Deleted

Updated Records:
ID: 101, Name: Rahul, Position: Manager, Salary: ₹60000.0
ID: 103, Name: Arun, Position: Tester, Salary: ₹40000.0
PS C:\Users\Madhu\Desktop\cts java\dsa\Employee Management System-4> 

```

Analysis

Time Complexity

Operation	Complexity
Add Employee	O(1)
Search Employee	O(n)
Traverse Employees	O(n)
Delete Employee	O(n)

Where **n** = number of employees.

Add Operation

employees[count] = e;

Direct insertion at the end.

Time Complexity = $O(1)$

Search Operation

for(...)

May need to check every employee.

Best Case = $O(1)$

Worst Case = $O(n)$

Traverse Operation

Visits all employees.

$O(n)$

Delete Operation

After finding the employee, remaining elements must be shifted.

$O(n)$

Limitations of Arrays:

1. Arrays have a fixed size and cannot be resized easily after creation.
2. Insertion and deletion operations are time-consuming because elements need to be shifted.
3. Memory may be wasted if the allocated array size is larger than the actual data stored.

When to Use Arrays?

Use arrays when:

Number of records is known in advance

Fast index-based access is required

Memory efficiency is important

Data size is small or fixed

- Examples: Student records

Exercise 5: Task Management System

1. Understanding Linked Lists

A **Linked List** is a linear data structure where elements (nodes) are connected using pointers

Each node contains:

- Data
- Reference to the next node

Singly Linked List

Each node points to the next node.

Task1 → Task2 → Task3 → NULL

Features

Less memory than doubly linked list

Easy insertion and deletion

Can only traverse forward

Doubly Linked List

Each node contains:

- Previous pointer
- Next pointer

NULL ← Task1 ⇄ Task2 ⇄ Task3 → NULL

Features

Forward traversal

Backward traversal

Easier deletion

Extra memory needed for previous pointer

2. Setup

Task.java

```
class Task {  
  
    int taskId;  
    String taskName;  
    String status;  
}
```

```

Task(int taskId, String taskName, String status) {
    this.taskId = taskId;
    this.taskName = taskName;
    this.status = status;
}
}

```

3. Implementation

TaskNode.java

```

class TaskNode {

    Task task;
    TaskNode next;

    TaskNode(Task task) {
        this.task = task;
        this.next = null;
    }
}

```

TaskManagement.java

```

class TaskManagement {

    TaskNode head = null;

    // Add Task
    void addTask(Task task) {

        TaskNode newNode = new TaskNode(task);

        if (head == null) {
            head = newNode;
            return;
        }

        TaskNode temp = head;

        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    // Search Task
    Task searchTask(int id) {

        TaskNode temp = head;

        while (temp != null) {

```

```

        if (temp.task.taskId == id) {
            return temp.task;
        }

        temp = temp.next;
    }

    return null;
}

// Traverse Tasks
void traverseTasks() {

    TaskNode temp = head;

    while (temp != null) {

        System.out.println(
            "Task ID: " + temp.task.taskId +
            ", Name: " + temp.task.taskName +
            ", Status: " + temp.task.status
        );

        temp = temp.next;
    }
}

// Delete Task
void deleteTask(int id) {

    if (head == null)
        return;

    if (head.task.taskId == id) {
        head = head.next;
        System.out.println("Task Deleted");
        return;
    }

    TaskNode temp = head;

    while (temp.next != null &&
        temp.next.task.taskId != id) {

        temp = temp.next;
    }

    if (temp.next == null) {
        System.out.println("Task Not Found");
        return;
    }

    temp.next = temp.next.next;

    System.out.println("Task Deleted");
}
}

```


Main.java

```
public class Main {

    public static void main(String[] args) {

        TaskManagement tm = new TaskManagement();

        tm.addTask(
            new Task(101,
                "Design UI",
                "Pending"));

        tm.addTask(
            new Task(102,
                "Develop Backend",
                "In Progress"));

        tm.addTask(
            new Task(103,
                "Testing",
                "Pending"));

        System.out.println("All Tasks:");

        tm.traverseTasks();

        System.out.println("\nSearching Task 102:");

        Task t = tm.searchTask(102);

        if (t != null) {

            System.out.println(
                "Task ID: " + t.taskId +
                ", Name: " + t.taskName +
                ", Status: " + t.status);
        }
        else {
            System.out.println("Task Not Found");
        }

        System.out.println("\nDeleting Task 102");

        tm.deleteTask(102);

        System.out.println("\nRemaining Tasks:");

        tm.traverseTasks();
    }
}
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\Madhu\Desktop\cts java\dsa\Task Management System-5> javac *.java
● PS C:\Users\Madhu\Desktop\cts java\dsa\Task Management System-5> java Main
All Tasks:
Task ID: 101, Name: Design UI, Status: Pending
Task ID: 102, Name: Develop Backend, Status: In Progress
Task ID: 103, Name: Testing, Status: Pending

Searching Task 102:
Task ID: 102, Name: Develop Backend, Status: In Progress

Deleting Task 102
Task Deleted

Remaining Tasks:
Task ID: 101, Name: Design UI, Status: Pending
Task ID: 103, Name: Testing, Status: Pending
❖ PS C:\Users\Madhu\Desktop\cts java\dsa\Task Management System-5> |
```

4. Analysis

Time Complexity

Operation	Time Complexity
Add Task	$O(n)$
Search Task	$O(n)$
Traverse Tasks	$O(n)$
Delete Task	$O(n)$

Where **n** = number of tasks.

Add Task

while(temp.next != null)

Need to reach the last node.

$O(n)$

Search Task

May check all nodes.

Best Case : $O(1)$

Worst Case : $O(n)$

Traverse Task

Visits every node.

$O(n)$

Delete Task

Need to locate the node first.

$O(n)$

Advantages of Linked Lists Over Arrays

Linked List	Array
Dynamic size	Fixed size
Easy insertion/deletion	Requires shifting elements
No memory wastage	May waste unused space
Memory allocated as needed	Memory allocated in advance

Exercise 6: Library Management System

1. Understanding Search Algorithms

Searching is the process of finding a specific element in a collection of data.

In a **Library Management System**, users may search for books by title or author.

Linear Search

Working

- Start from the first element.
- Compare each element with the target.
- Stop when the element is found or the list ends.

Example

Books:

Java

Python

C++

Data Structures

Search: C++

Java -No

Python -No

C++ -Yes

Characteristics

Works on sorted and unsorted data

Easy to implement

Slow for large datasets

Binary Search

Working

- Works only on sorted data.
- Find the middle element.
- If target equals middle → found.
- If target is smaller → search left half.
- If target is larger → search right half.
- Repeat until found.
-

Example

Books:

C++

Data Structures

Java

Python

Search: Java

Middle = Data Structures

Java > Data Structures

Search Right Half

Middle = Java

Found

Characteristics

Very fast

Suitable for large datasets

Requires sorted data

2. Setup

Book.java

```
class Book {  
  
    int bookId;  
    String title;  
    String author;  
  
    Book(int bookId, String title, String author) {  
        this.bookId = bookId;  
        this.title = title;  
        this.author = author;  
    }  
  
    public String toString() {  
        return "Book ID: " + bookId +  
            ", Title: " + title +  
            ", Author: " + author;  
    }  
}
```

3. Implementation

LibrarySearch.java

```

public class LibrarySearch {

    // Linear Search by Title
    static Book linearSearch(Book[] books, String targetTitle) {

        for (Book book : books) {

            if (book.title.equalsIgnoreCase(targetTitle)) {
                return book;
            }
        }

        return null;
    }

    // Binary Search by Title
    static Book binarySearch(Book[] books, String targetTitle) {

        int low = 0;
        int high = books.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            int compare =
                books[mid].title.compareToIgnoreCase(targetTitle);

            if (compare == 0)
                return books[mid];

            else if (compare < 0)
                low = mid + 1;

            else
                high = mid - 1;
        }

        return null;
    }

    public static void main(String[] args) {

        // Sorted by title
        Book[] books = {

            new Book(101,
                "C++ Programming",
                "Bjarne Stroustrup"),

            new Book(102,
                "Data Structures",
                "Mark Allen Weiss"),

            new Book(103,
                "Java Programming",
                "James Gosling"),
        }
    }
}

```

```

        new Book(104,
            "Python Basics",
            "Guido van Rossum")
    };

    String searchTitle = "Java Programming";

    System.out.println("Linear Search:");

    Book result1 =
        linearSearch(books, searchTitle);

    if (result1 != null)
        System.out.println(result1);
    else
        System.out.println("Book Not Found");

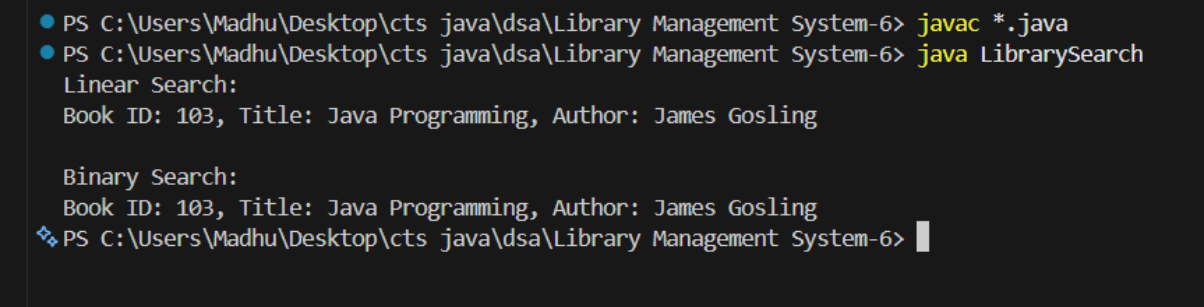
    System.out.println("\nBinary Search:");

    Book result2 =
        binarySearch(books, searchTitle);

    if (result2 != null)
        System.out.println(result2);
    else
        System.out.println("Book Not Found");
    }
}

```

Output:



```

PS C:\Users\Madhu\Desktop\cts java\dsa\Library Management System-6> javac *.java
PS C:\Users\Madhu\Desktop\cts java\dsa\Library Management System-6> java LibrarySearch
Linear Search:
Book ID: 103, Title: Java Programming, Author: James Gosling

Binary Search:
Book ID: 103, Title: Java Programming, Author: James Gosling
PS C:\Users\Madhu\Desktop\cts java\dsa\Library Management System-6>

```

4. Analysis

Time Complexity Comparison

Case	Linear Search	Binary Search
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$

When to Use Linear Search?

Use Linear Search when:

Data is unsorted

Dataset is small

Simplicity is preferred

Example:

10–50 books

Linear search is sufficient.

When to Use Binary Search?

Use Binary Search when:

Data is sorted

Dataset is very large

Fast searching is required

Example:

Thousands or millions of books

Binary search is the better choice.

Exercise 7: Financial Forecasting

Recursion:

Recursion is a programming technique where a method calls itself to solve a smaller version of the same problem.

A recursive function consists of:

1. **Base Case** – Stops the recursion.
2. **Recursive Case** – Calls itself with a smaller input.

Example: Factorial

```
int factorial(int n) {  
    if(n == 0)  
        return 1;  
  
    return n * factorial(n - 1);  
}
```

Why Recursion Helps?

Recursion simplifies problems that can be broken into smaller subproblems.

Examples:

- Tree Traversal
- Binary Search
- Fibonacci Series
- Financial Forecasting

2. Setup

Suppose:

- Current Investment = ₹10,000
- Annual Growth Rate = 10%
- Forecast Period = 5 Years

Formula:

Future Value = Current Value $\times$ (1 + Growth Rate)

Recursive Relation:

$FV(n) = FV(n-1) \times (1 + \text{rate})$

Base Case:

$FV(0)$ = Current Value

3. Implementation

FinancialForecast.java

```
public class FinancialForecast {

    static double predictFutureValue(
        double currentValue,
        double growthRate,
        int years) {

        // Base Case
        if (years == 0)
            return currentValue;

        // Recursive Case
        return predictFutureValue(
            currentValue,
            growthRate,
            years - 1)
            * (1 + growthRate);
    }

    public static void main(String[] args) {

        double currentValue = 10000;
        double growthRate = 0.10; // 10%
        int years = 5;

        double futureValue =
            predictFutureValue(
                currentValue,
                growthRate,
                years);

        System.out.printf(
            "Future Value after %d years = ₹%.2f",
            years,
            futureValue);
    }
}
```

Output:

```
PS C:\Users\Madhu\Desktop\cts java\dsa\Library Management System-7> java FinancialForecast
Future Value after 5 years = ₹16105.10
PS C:\Users\Madhu\Desktop\cts java\dsa\Library Management System-7> █
```

4. Analysis

Time Complexity

Let:

n = number of years

Each recursive call performs:

- One multiplication
- One recursive call

Therefore:

$$T(n) = T(n-1) + O(1)$$

Result:

$$\text{Time Complexity} = O(n)$$

Space Complexity

Because each recursive call is stored on the call stack:

$$\text{Space Complexity} = O(n)$$

Recursive Call Stack Example

```
predict(5)
predict(4)
predict(3)
predict(2)
predict(1)
predict(0)
```

Total stack frames: 6

Hence: $O(n)$ space

Optimization

Problem with Some Recursive Solutions

Consider Fibonacci:

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2);$$

This recalculates values repeatedly.

Example:

$\text{fib}(5)$

fib(4)
fib(3)

fib(3) repeated
fib(2) repeated

This causes:

$O(2^n)$

complexity.

Optimization 1: Memoization

Store already calculated results. Reuse previous calculations instead of recomputing.

Optimization 2: Dynamic Programming

Build solutions from smaller results iteratively.

```
for(int i=1; i<=years; i++)  
    value *= (1 + growthRate);
```

Complexity:

Time = $O(n)$

Space = $O(1)$

Optimization 3: Direct Formula

Financial forecasting can be calculated directly using compound growth:

Future Value =
Current Value $\times (1 + \text{Rate})^{\text{Years}}$

Java:

```
double futureValue =  
    currentValue *  
    Math.pow(1 + growthRate, years);
```

Example: `double fv = 10000 * Math.pow(1.10, 5);`

Complexity: Time Complexity $\approx O(\log n)$